WEB APPLICATION VULNERABILITY SCANNER

Cybersecurity Internship Project Report

Organization: Elevate Labs

Project Domain: Cybersecurity

Submitted by: Gandhari Keerthi

Technology Stack: Python, Requests, Visual Studio Code, Git, GitHub

1. INTRODUCTION

Web application security is an essential aspect of protecting online services against potential security weaknesses. Misconfigured security headers and other implementation issues may increase exposure to certain web-based risks. Identifying and documenting these observations is an important part of an initial security assessment.

The Web Application Vulnerability Scanner is a Python-based educational project developed to automate selected web security checks on authorized targets. The project demonstrates the application of Python programming and HTTP communication to collect security-related observations and produce a readable scan report.

2. ABSTRACT

This project implements a lightweight, Python-based tool for preliminary web application security assessment. Using the Requests library, the scanner communicates with a specified website and examines selected response headers and other checks implemented in the program. It summarizes the collected observations, assigns preliminary priority levels according to its implemented logic, and saves the output to a text file named "scan_report.txt".

The project provides hands-on experience in security automation, HTTP response analysis, exception handling, and report generation. It also demonstrates source-code management and project documentation through Git and GitHub. The scanner is intended as a learning tool; its findings require manual verification and should not be interpreted as confirmation of exploitable vulnerabilities.

3. TOOLS AND TECHNOLOGIES USED

- Python 3: Core programming language used to implement the scanner.

- Requests: Python library used to send HTTP requests and process website responses.

- Visual Studio Code: Development and execution environment.

- Git: Version control and change management.

- GitHub: Source-code hosting and project documentation.

- Windows: Development environment.

4. PROJECT IMPLEMENTATION

The project was developed through the following stages:

1. Environment Setup: Prepared the Python development environment and configured the project workspace in Visual Studio Code.

2. Scanner Development: Implemented the scanning logic in "scanner.py" using Python and the Requests library.

3. HTTP Response Analysis: Sent requests to an authorized target and examined selected response information.

4. Security Header Review: Checked for selected security-related HTTP headers and recorded observations requiring further review.

5. Finding Classification: Summarized observations and assigned preliminary priority categories using the program's implemented logic.

6. Report Generation: Displayed the scan output and saved the results in "scan_report.txt", including a summary of selected checks.

7. Testing and Documentation: Executed the program, reviewed its output, and uploaded the source code, report, and README documentation to GitHub.

Observed Test Output: In one captured execution, the report recorded two low-priority findings, five headers needing review, and zero forms found. These are run-specific observations and do not independently establish the presence or absence of vulnerabilities.

Project Repository:

https://github.com/keerthi-project/Web-Application-Vulnerability-Scanner

5. CONCLUSION AND FUTURE SCOPE

The project provided practical experience in Python-based security automation, HTTP communication, response-header inspection, exception handling, and text-based report generation. It also strengthened familiarity with version control, GitHub repository management, and technical documentation. Future enhancements could include expanded security checks with documented detection criteria, improved input validation, automated testing, structured report formats, and more comprehensive error handling. Any additional functionality should be validated against authorized test environments before being used for security assessment.